

Tokenization и Embeddings

Минеев Анатолий РИ-511055

1. Токенизация

Токенизация

Токенизация — это процесс разбиения текста на более мелкие единицы (токены)

Виды:

- | | |
|--------------------|--|
| 1) Character-level | <code>["T", "o", "k", "e", "n", "i", "z", "a", "t", "i", "o", "n", "..."]</code> |
| 2) Word-level | <code>["Tokenization", "is", "fascinating", "!"]</code> |
| 3) Subword-level | <code>["Token", "ization", "is", "fasc", "inating", "!"]</code> |

BPE

Byte-Pair Encoding

GPT, GPT-2, RoBERTa, BART, DeBERTa

Алгоритм:

1. Анализируется весь обучающий корпус текстов. Изначально алфавит состоит из всех байт.
2. Алгоритм считает, какая пара токенов встречается вместе чаще всего, добавляет её в словарь и заменяет в корпусе.
3. Повторяем шаг 2, пока словарь не достигнет заданного размера

Пример работы ВРЕ

«Мой дядя самых честных правил»

d0 9c d0 be d0 b9 20 d0 b4 d1 8f d0 b4 d1 8f 20 d1 81 d0 b0 d0 bc d1 8b d1 85 20 d1 87 d0 b5 d1
81 d1 82 d0 bd d1 8b d1 85 20 d0 bf d1 80 d0 b0 d0 b2 d0 b8 d0 bb

Словарь: 00 01 ... fe ff (256)

Пример работы ВРЕ

«Мой дядя самых честных правил»

d0 9c d0 be d0 b9 20 d0 b4 d1 8f d0 b4 d1 8f 20 d1 81 d0 b0 d0 bc d1 8b d1 85 20 d1 87 d0 b5 d1
81 d1 82 d0 bd d1 8b d1 85 20 d0 bf d1 80 d0 b0 d0 b2 d0 b8 d0 bb

Словарь: 00 01 ... fe ff (256)

Находим самую частую пару

Пример работы ВРЕ

«Мой дядя самых честных правил»

d0 9c d0 be d0 b9 20d0 b4 d1 8f d0 b4 d1 8f 20 d1 81 d0 b0 d0 bc d1 8b d1 85 20 d1 87 d0 b5 d1
81 d1 82 d0 bd d1 8b d1 85 20d0 bf d1 80 d0 b0 d0 b2 d0 b8 d0 bb

Словарь: 00 01 ... fe ff 20d0 (257)

Добавляем эту пару в словарь и заменяем ей исходные токены

Пример работы ВРЕ

«Мой дядя самых честных правил»

d0 9c d0 be d0 b9 20d0 b4d18f d0 b4d18f 20d1 81 d0b0d0 bc d18bd1 85 20d1 87 d0 b5 d1 81 d1 82 d0
bd d18bd1 85 20d0 bf d1 80 d0b0d0 b2 d0 b8 d0 bb

Словарь: 00 01 ... fe ff 20d0 20d1 b4d1 d0b0 b4d18f d0b0d0 d18b d18bd1 (264)

Пример работы ВРЕ

«Судей решительных и строгих»

d0 a1 d1 83 d0 b4 d0 b5 d0 b9 20 d1 80 d0 b5 d1 88 d0 b8 d1 82 d0 b5 d0 bb d1 8c d0 bd d1 8b d1
85 20 d0 b8 20 d1 81 d1 82 d1 80 d0 be d0 b3 d0 b8 d1 85

Словарь: 00 01 ... fe ff 20d0 20d1 b4d1 d0b0 b4d18f d0b0d0 d18b d18bd1 (264)

Пример работы ВРЕ

«Судей решительных и строгих»

d0 a1 d1 83 d0 b4 d0 b5 d0 b9 20d1 80 d0 b5 d1 88 d0 b8 d1 82 d0 b5 d0 bb d1 8c d0 bd d18bd1 85
20 d0 b8 20d1 81 d1 82 d1 80 d0 be d0 b3 d0 b8 d1 85

Словарь: 00 01 ... fe ff 20d0 20d1 b4d1 d0b0 b4d18f d0b0d0 d18b d18bd1 (264)

WordPiece

BERT

Алгоритм (аналогичен BPE, но с другой метрикой):

- Вместо подсчета частоты пары, алгоритм вычисляет, какое объединение максимально увеличит вероятность языковой модели на обучающем корпусе.
- Формула оценки: $\text{score} = P(AB) / (P(A) * P(B))$. Если A и B часто идут вместе, их объединение будет сильно повышать score.

Пример работы WordPiece

«Мой дядя самых честных правил»

М ##о ##й д ##я ##д ##я с ##а ##м ##ы ##х ч ##е ##с ##т ##н ##ы ##х п ##р ##а ##в ##и ##л

Словарь: М ##о ##й д ##я ##д с ##а ##м ##ы ##х ч ##е ##с ##т ##н п ##р ##в ##и ##л
(21)

Пример работы WordPiece

«Мой дядя самых честных правил»

М ##о ##й д ##я ##д ##я с ##а ##м ##ы ##х ч ##е ##с ##т ##н ##ы ##х п ##р ##а ##в ##и ##л

Словарь: М ##о ##й д ##я ##д с ##а ##м ##ы ##х ч ##е ##с ##т ##н п ##р ##в ##и ##л ...

$$\text{score}(M, \text{##о}) = \frac{7}{23 \cdot 1477} = 0.0002$$

$$\text{score}(\text{##ы}, \text{##х}) = \frac{47}{385 \cdot 140} = 0.0008$$

$$\text{score}(\text{##д}, \text{##я}) = \frac{7}{416 \cdot 244} = 0.00007$$

Unigram

ALBERT, T5, mBART, Big Bird и XLNet

Обратная логика: берём огромный начальный словарь и сокращаем его

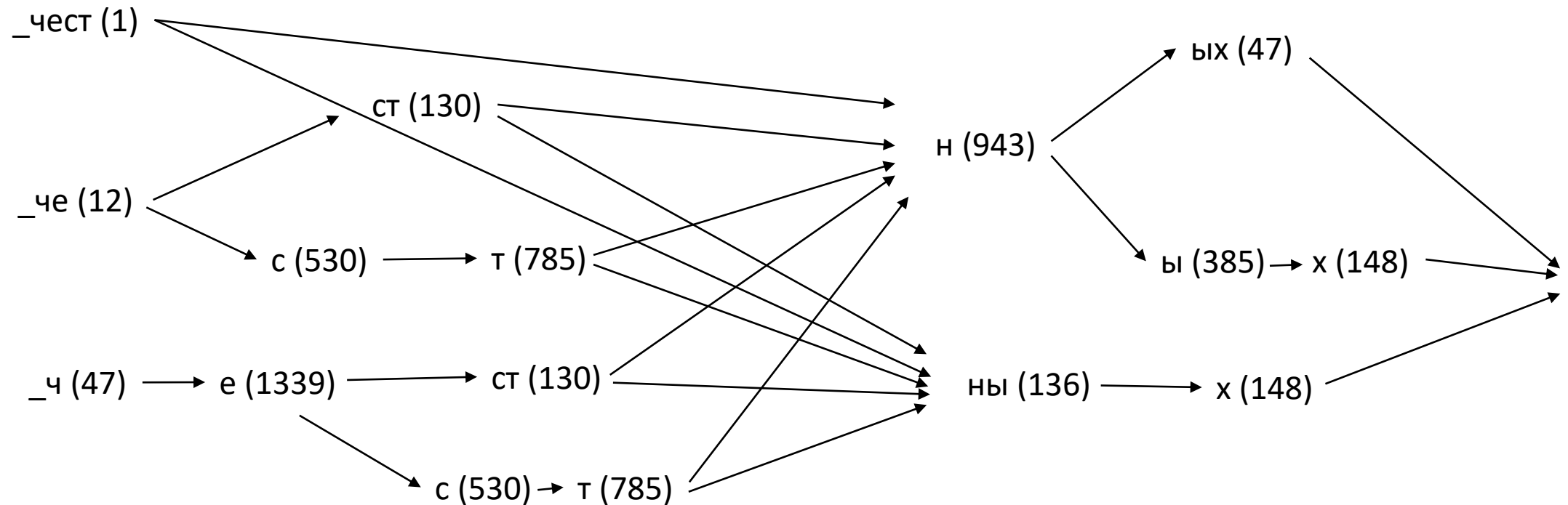
Алгоритм:

1. Инициализируем словарь (например, берём наиболее частые подстроки или применяем BPE с большим размером словаря)
2. Выбираем токены, исключение которых приведёт к наименьшим потерям (10 – 20% от размера словаря). Исключаем их из словаря.
3. Повторяем шаг 2, пока словарь не достигнет заданного размера

Unigram: токенизация

Рассмотрим токенизацию слова «_честных»

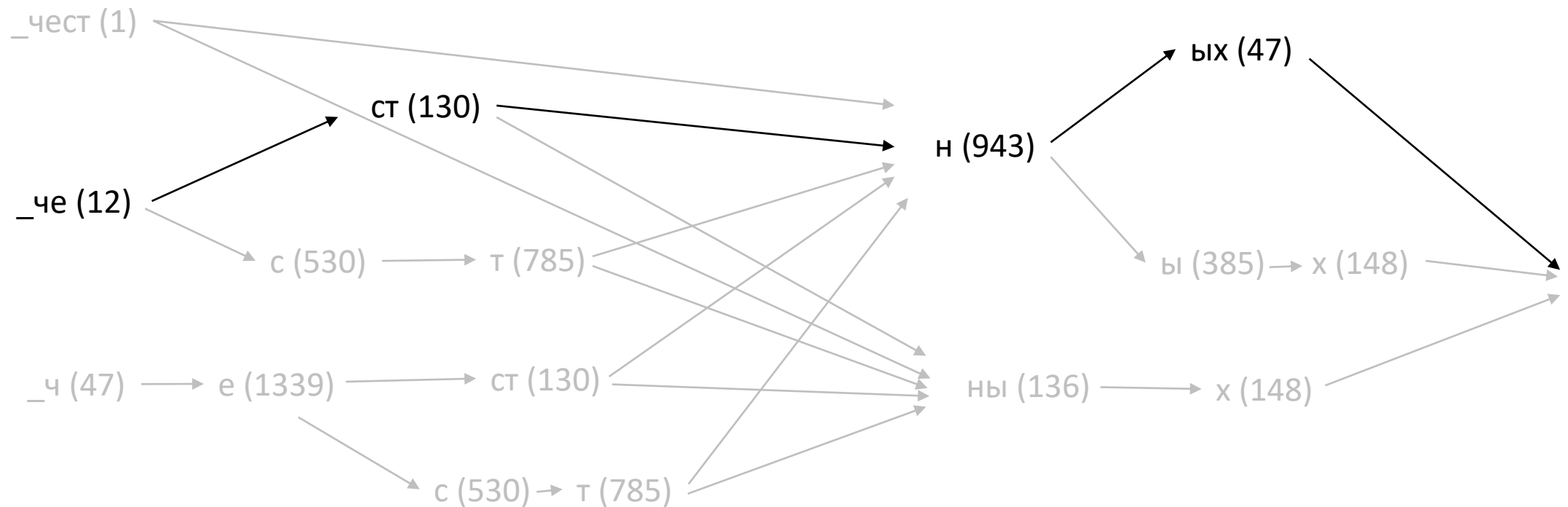
Словарь: а ... я, _чест, ых, _че, ст, ны



Unigram: токенизация

Строим граф всех возможных вариантов токенизации и считаем score

$$score = \frac{12}{10000} \cdot \frac{130}{10000} \cdot \frac{943}{10000} \cdot \frac{47}{10000} = 6,91 \cdot 10^{-13}$$

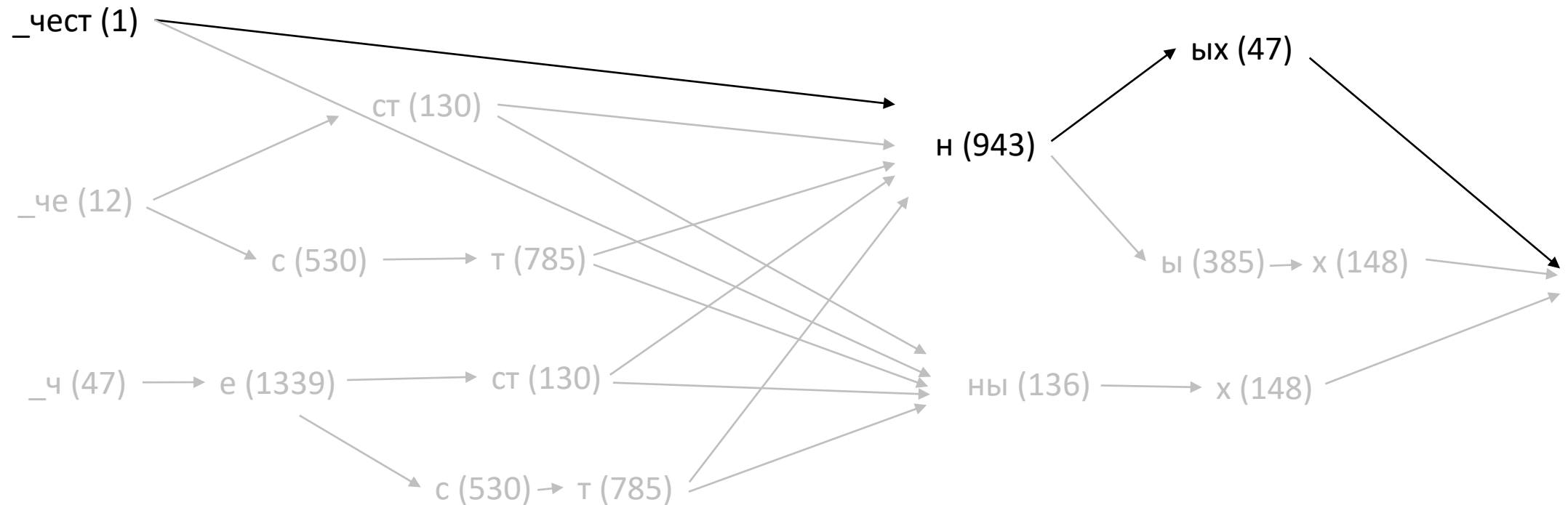


Unigram: токенизация

Выбираем вариант с самым большим score

`_честных = _чест + н + ых`

$$score = \frac{1}{10000} \cdot \frac{943}{10000} \cdot \frac{47}{10000} = 4,43 \cdot 10^{-11}$$

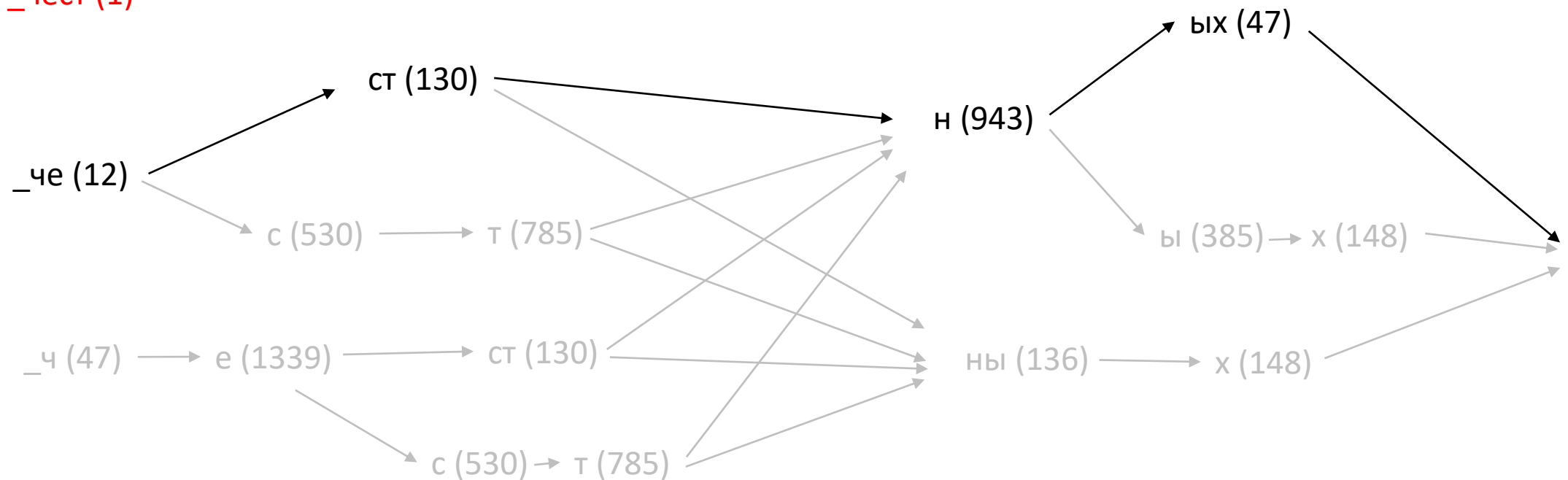


Unigram: обучение

Если мы уберём токен «_чест» из словаря, то потери составят:

$$-(-\log(4,43 \cdot 10^{-11})) + (-\log(6,91 \cdot 10^{-13})) = 1,807$$

_чест (1)

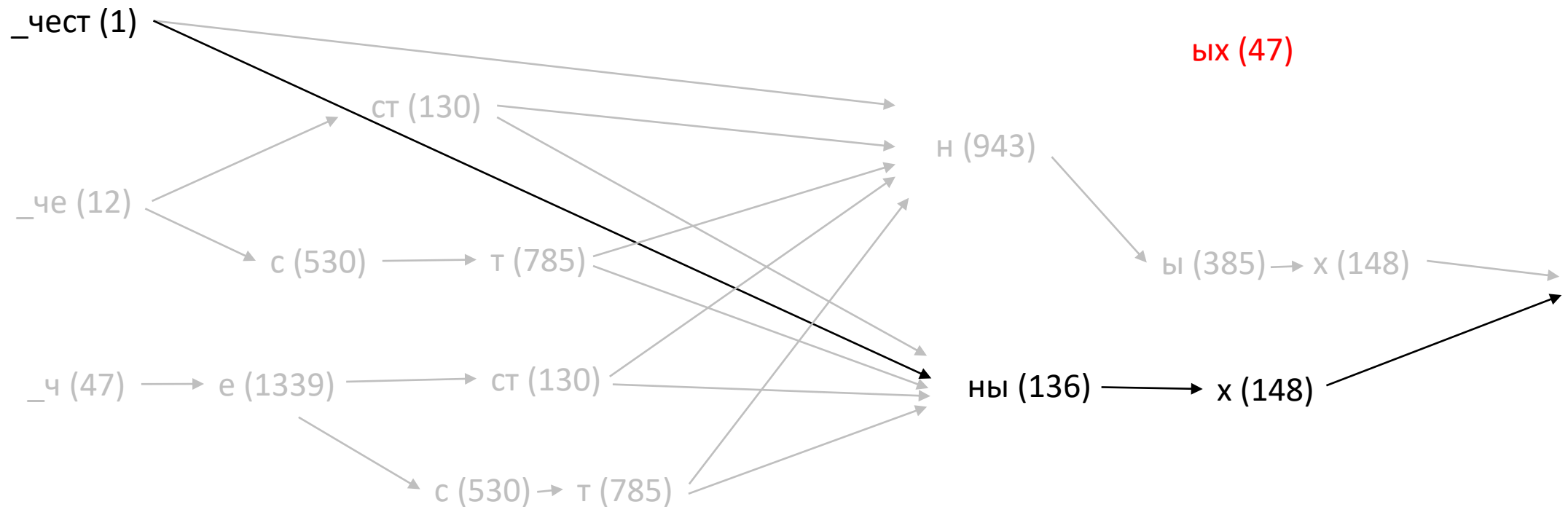


Unigram: обучение

Если мы уберём токен «ых» из словаря, то потери составят:

$$-(-\log(4,43 \cdot 10^{-11})) + (-\log(2,01 \cdot 10^{-11})) = 0,34$$

Это меньше, чем потери от удаления токена «_чест», значит удаляем токен «ых»



Сравнение токенизаторов

«Мой дядя самых честных правил»

BPE:

Tokens: ['ĠÐ¼Ð¼Ð, ', 'ĠÐ´', 'Ñ½Ð´', 'Ñ½', 'ĠÑġÐ°Ð¼', 'Ñ½Ñ½ĥ', 'ĠÑ½', 'ÐµÑġÑ½Ĥ', 'Ð¼Ñ½Ñ½ĥ', 'ĠÐ½ÑġÐ°Ð²', 'Ð, Ð»']

Token Count: 11

WordPiece:

Tokens: ['мои', 'д', '##я', '##дя', 'сам', '##ых', 'ч', '##ест', '##ных', 'пра', '##ви', '##л']

Token Count: 12

Unigram:

Tokens: ['_мо', 'и', '_д', 'я', 'д', 'я', '_сам', 'ых', '_', 'че', 'ст', 'ных', '_прав', 'ил']

Token Count: 14

Сравнение токенизаторов

«Судей решительных и строгих 😊»

BPE:

Tokens: ['ĠÑġ', 'ÑĥĐ´', 'ĐμĐ.', 'ĠÑĠĐμÑĪ', 'Đ,ÑĤĐμĐ»ÑĬ', 'Đ½ÑĩÑĥ', 'ĠĐ.', 'ĠÑġÑĤÑĠ', 'Đ¾Đ³', 'Đ,Ñĥ', 'Ġ', '[UNK]', 'ĸ', '[UNK]', '[UNK]']

Token Count: 15

WordPiece:

Tokens: ['су', '##деи', 'ре', '##ши', '##тель', '##ных', 'и', 'ст', '##ро', '##ги', '##х', '[UNK]']

Token Count: 12

Unigram:

Tokens: ['—су', 'де', 'и', '—решительн', 'ых', '—и', '—стро', 'г', 'их', '—', '😊']

Token Count: 11

2. Векторизация

One-Hot Encoding

Словарь: Мой, дядя, самых, честных, правил, когда, не, ...

[1, 0, 0, 0, 0, 0, ..., 0]

[0, 1, 0, 0, 0, 0, ..., 0]

[0, 0, 1, 0, 0, 0, ..., 0]

...

[0, 0, 0, 0, 0, 0, ..., 1]

- Просто в реализации
- Большой размер векторов
- Нет семантики

Word2Vec: обучение

Мой дядя **самых** честных правил когда не в шутку ...

Самых мой
Самых дядя
Самых честных
Самых правил

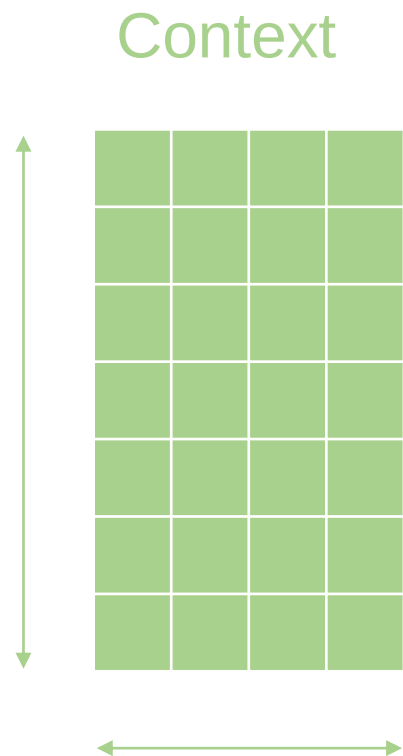
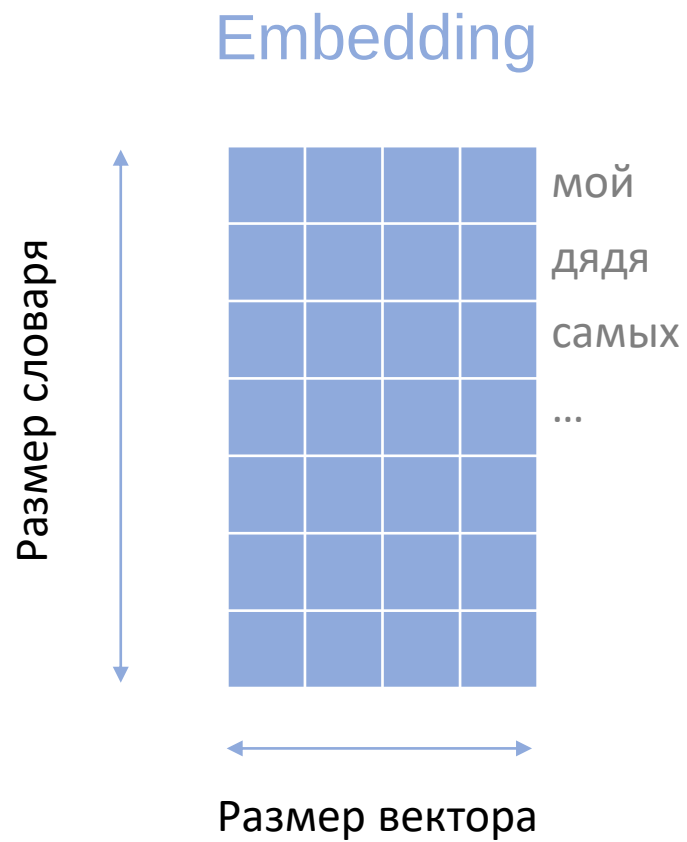
Мой дядя самых **честных** правил когда не в шутку ...

Честных дядя
Честных самых
Честных правил
Честных когда

Мой дядя самых честных **правил** когда не в шутку ...

Правил самых
Правил честных
Правил когда
Правил не

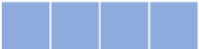
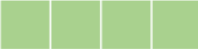
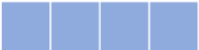
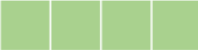
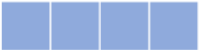
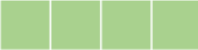
Word2Vec: обучение



Word2Vec: обучение

Входное слово	Контекст	класс
Самых	мой	1
Самых	Онегин	0
Самых	судей	0
Самых	дядя	1

Word2Vec: обучение

Входное слово	Контекст	класс	*	Sigmoid()	Error
Самых 	мой 	1	0.2	0.55	0.45
Самых 	Онегин 	0	0.74	0.68	0.68
Самых 	судей 	0	-1.11	0.25	0.25

Word2Vec: итоги

```
words = model.wv.most_similar("реприв", topn=5)

for i in range(len(words)):
    print(f'{i}) {words[i][0]} ({round(words[i][1], 3)})')
```

✓ 0.5s

- 0) дирижёр (0.778)
- 1) башмет (0.763)
- 2) оркестр (0.718)
- 3) симфонический (0.703)
- 4) спиваков (0.686)

```
words = model.wv.most_similar("машина", topn=5)

for i in range(len(words)):
    print(f'{i}) {words[i][0]} ({round(words[i][1], 3)})')
```

✓ 0.0s

- 0) автомобиль (0.879)
- 1) автомашина (0.824)
- 2) джип (0.793)
- 3) уаз (0.75)
- 4) микроавтобус (0.747)

```
words = model.wv.most_similar("ильич", topn=5)

for i in range(len(words)):
    print(f'{i}) {words[i][0]} ({round(words[i][1], 3)})')
```

[35] ✓ 0.0s

- ... 0) ленин (0.722)
- 1) маяковский (0.569)
- 2) достоевский (0.56)
- 3) сталин (0.557)
- 4) вождь (0.554)

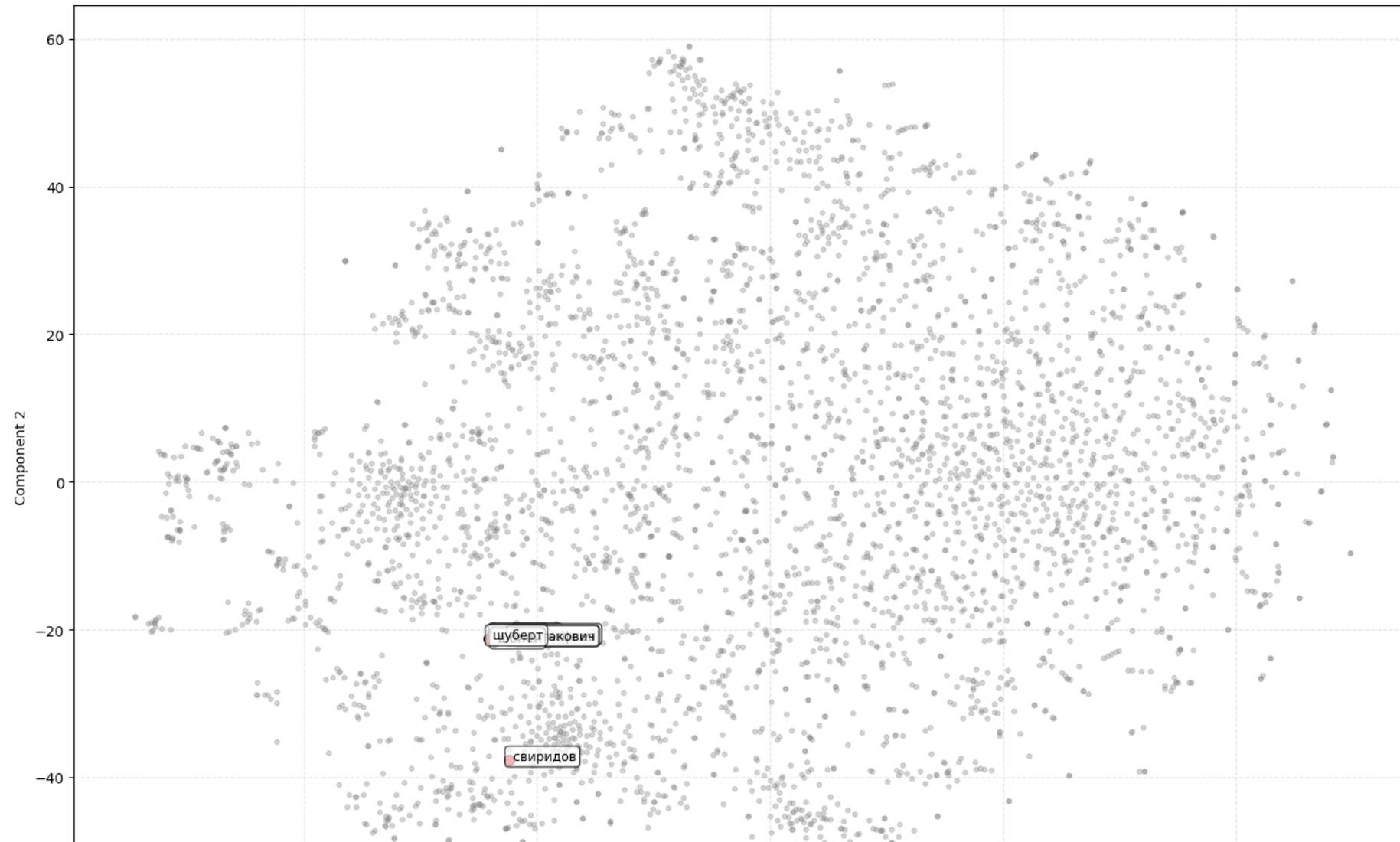
Word2Vec: итоги

```
# 2. Аналогии (проверка линейных свойств)
print(model.wv.most_similar(positive=["италия", "москва"], negative=["россия"], topn=3)) # Москва - Россия + Италия = Рим
[14] ✓ 0.0s
... [('рим', 0.734878420829773), ('мадрид', 0.7005112171173096), ('палермо', 0.6866071224212646)]

print(model.wv.most_similar(positive=["доллар", "россия"], negative=["сша"], topn=3)) # Доллар - США + Россия = Рубль
[15] ✓ 0.0s
... [('рублей', 0.7726427316665649), ('гривна', 0.6079665422439575), ('евро', 0.5958508849143982)]

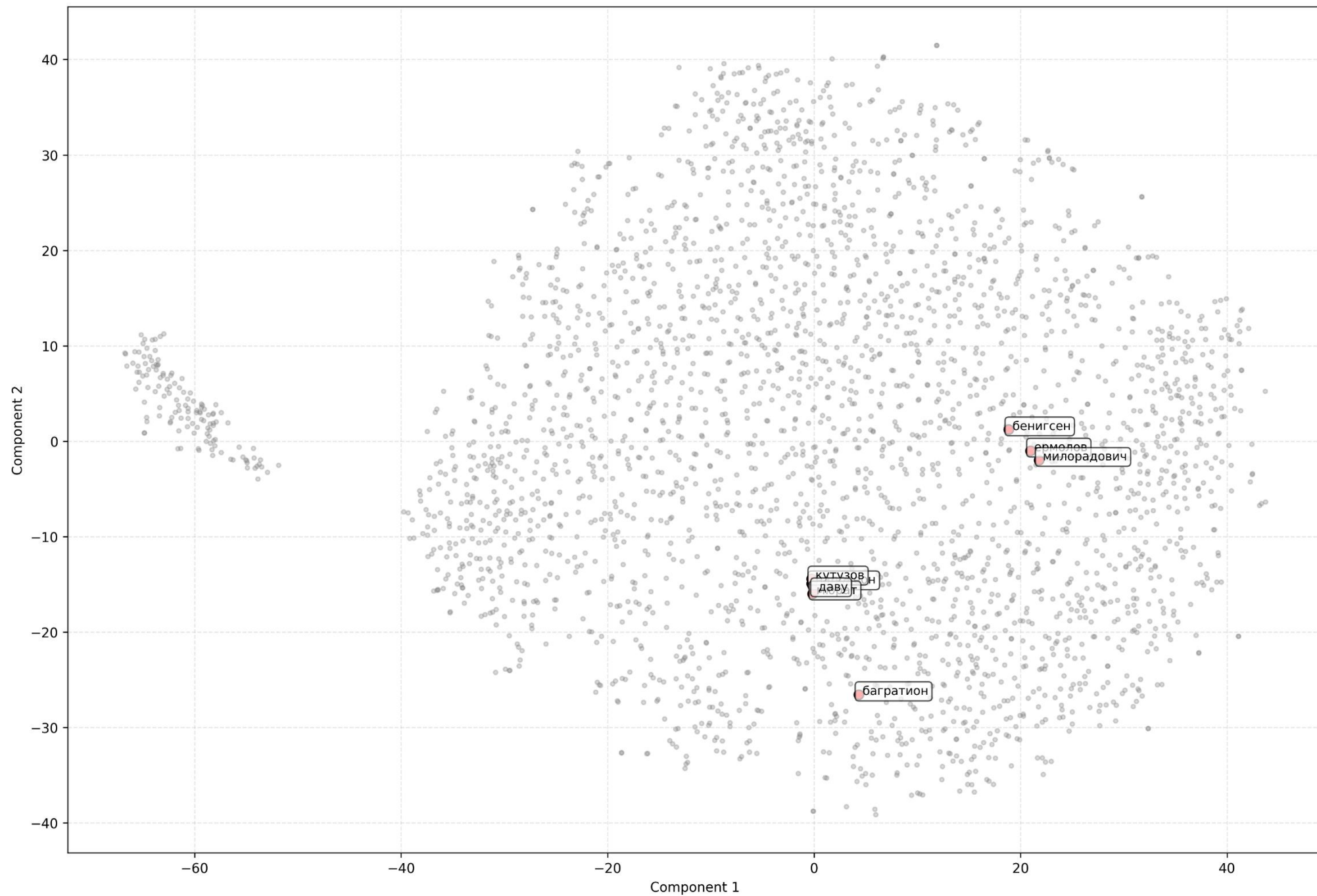
print(model.wv.most_similar(positive=["муж", "женщина"], negative=["мужчина"], topn=3)) # муж - мужчина + женщина = жена
[21] ✓ 0.0s
... [('жена', 0.829801082611084), ('дочь', 0.8250786662101746), ('супруг', 0.799607515335083)]
```

Word2Vec Embeddings (TSNE)

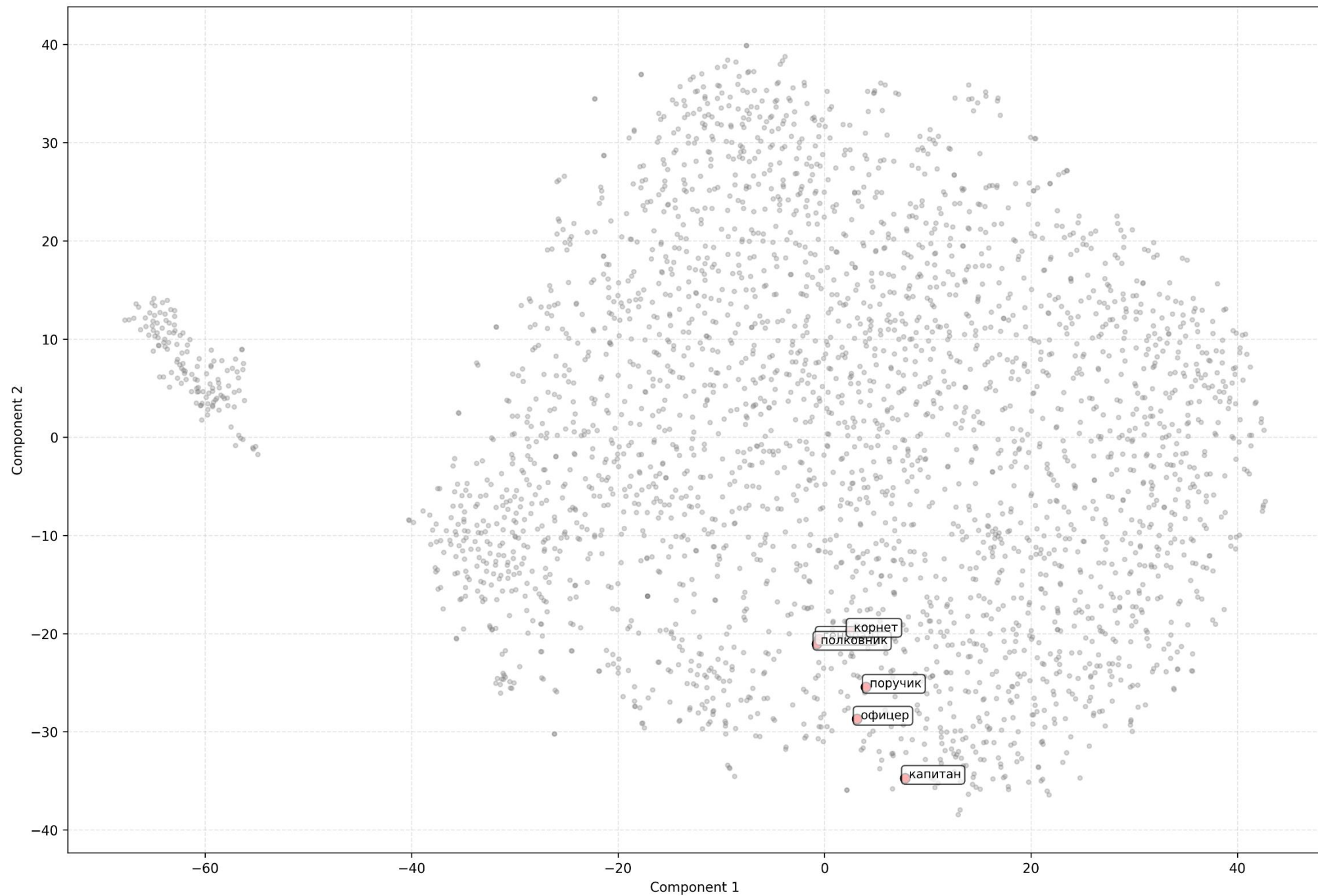


Шуберт, Шостакович, Чайковский, Прокофьев, Свиридов, Моцарт, Бетховен, Шопен

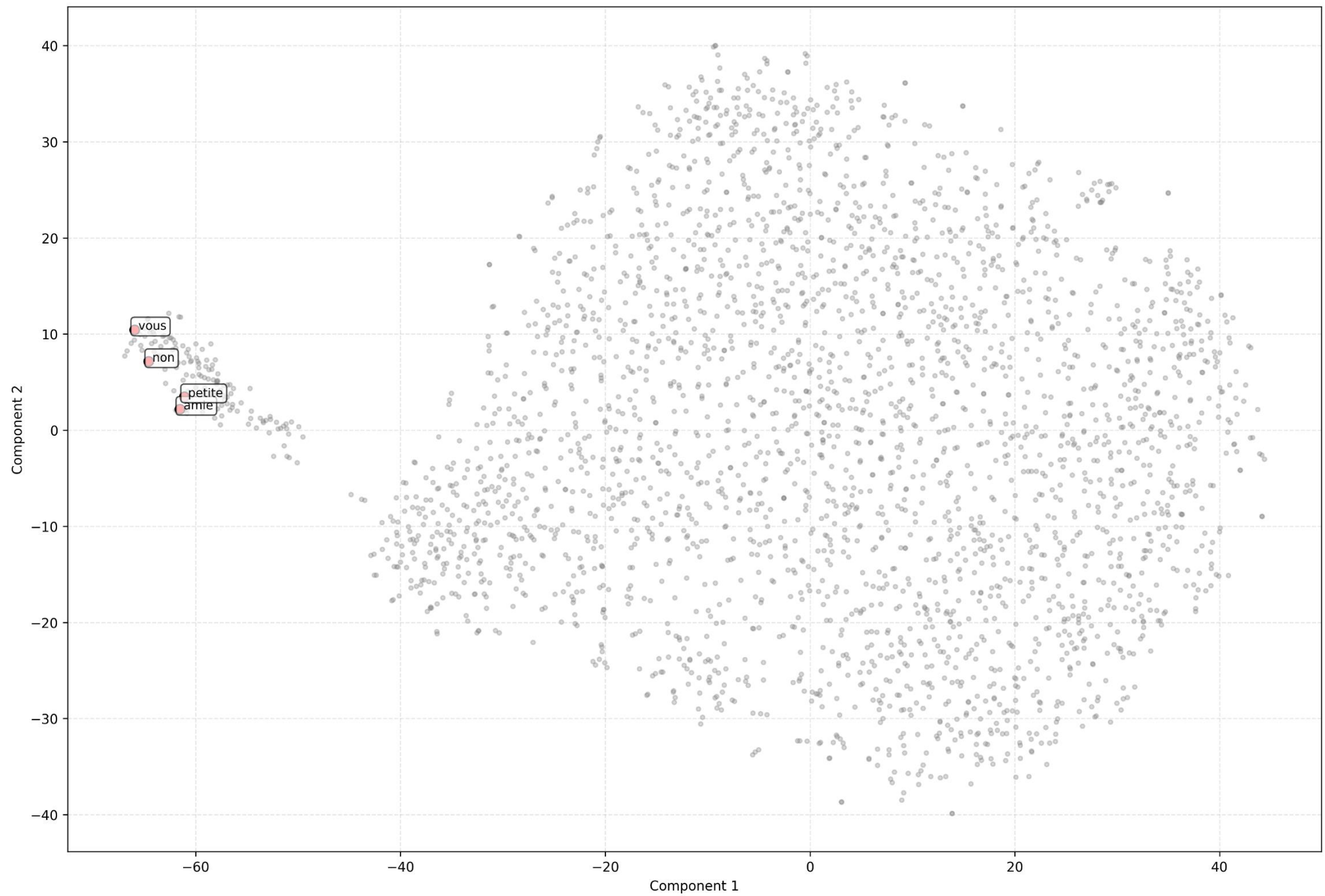
Word2Vec Embeddings (TSNE)



Word2Vec Embeddings (TSNE)



Word2Vec Embeddings (TSNE)



Более современные варианты

Учёт контекста

Архитектура: трансформер

Пример:

- BERT